

RealMythosReasoning: A Patch-Unaware Reasoning Distillation Dataset Grounded in Quality-Gated PoCs

Zongjie Li
zligo@connect.ust.hk
The Hong Kong University of Science
and Technology
Hong Kong, Hong Kong

Liwen Wang
lwanged@cse.ust.hk
The Hong Kong University of Science
and Technology
Hong Kong, Hong Kong

Chaozheng Wang
czwang23@cse.cuhk.edu.hk
The Chinese University of Hong Kong
Hong Kong, Hong Kong

Zimo Ji
zjiag@connect.ust.hk
The Hong Kong University of Science
and Technology
Hong Kong, Hong Kong

Xing Han
xhanbg@cse.ust.hk
The Hong Kong University of Science
and Technology
Hong Kong, Hong Kong

 Dataset: <https://huggingface.co/datasets/RealMythos/RealMythosReasoning>  Source Code: <https://github.com/tszdanger/RealMythos>

Abstract

Closed frontier LLMs can locate and exploit real C/C++ vulnerabilities at near-expert quality, yet open models lag far behind: no public dataset pairs real CVE code with reasoning and a runnable proof-of-concept (PoC). We introduce **RealMythosReasoning**, the first open dataset that jointly provides all three. The key idea is to separate teacher-only evidence (patch, CVE description, CWE label, context) from the student-visible task (vulnerable code only), so the teacher recovers the root cause while the released example forces code-grounded reasoning. A five-stage pipeline curates 9,964 CVE-unique vulnerabilities, generates traces and PoCs with DeepSeek-V4-Pro, strips patch and CVE references, and scores each PoC on a six-axis rubric. The release contains 6,159 SFT-ready records over 149 CWE classes with mean PoC quality 27.90/30; cross-judge replays with Gemini-3.1-Pro, Claude-Opus-4.6, and GPT-5.4 confirm the score is not judge-specific. Fine-tuning a Qwen3.5-9B student lifts its score on a 141-record held-out benchmark from 3/141 parsable outputs to 24.6/30, within 1 point of a 3× larger open model and 0.5 of the frontier closed DeepSeek-V4-Flash.

Keywords

cybersecurity reasoning, vulnerability analysis, proof-of-concept generation, reasoning distillation, supervised fine-tuning

1 Introduction

Recent progress in deep learning has substantially expanded the scope of automated security analysis. Large language models are now used for security triage, vulnerability explanation, code review, and incident response, and both open models and commercial security platforms are moving these capabilities into practical workflows [7, 14, 24]. At the same time, in mathematics and programming, distilling long reasoning traces from stronger teachers has become an effective way to improve model capability [1, 12, 13]. A natural question is whether a similar data-centric recipe can teach models to reason over real vulnerable code rather than only answer security knowledge questions.

The main obstacle is the lack of suitable public data. Existing cyber instruction and reasoning datasets are useful for concept

learning, certification-style questions, CTI mapping, and remediation advice, but they rarely describe the process by which a model should understand a concrete vulnerability and construct a proof-of-concept for it [4, 8, 14, 17, 25]. Vulnerability corpora provide real CVE-linked code and metadata, but they usually stop at labels, commits, patches, or function pairs [5, 9, 11, 15, 18, 23]. They do not provide the missing training object needed for reasoning distillation: a code-grounded trace that connects the vulnerable program state to a PoC-oriented output while avoiding leakage from privileged patch or CVE context. This gap is the starting point of this paper.

The central difficulty is not merely collecting vulnerable functions. At scale, the input to the dataset is source code drawn from large and heterogeneous repositories, while the desired output is a high-quality PoC-oriented response. Without privileged signals such as the patch and CVE description, even a strong teacher often misreads the root cause from the function alone. Our key insight is to treat dataset construction as a perspective-conversion problem. During generation, a teacher can be given privileged evidence such as patches, vulnerability descriptions, CWE labels, and augmented context in order to recover the root cause and the PoC design. During release, the training example must be converted back to the student view: the reasoning must anchor only on the visible vulnerable code, with patch- and CVE-dependent statements removed. This separation between the generation view and the learning view is the organizing principle of RealMythosReasoning.

We instantiate this principle in a multi-stage pipeline. The pipeline first curates a multi-source CVE corpus from PrimeVul, MegaVul, ReposVul, and CVEfixes [5, 11, 18, 23], then augments cases whose vulnerable function does not contain enough local context. A teacher model generates vulnerability reasoning and a PoC-oriented response under teacher-only evidence. The reasoning is then rewritten into a patch-unaware form, and each PoC is scored by a six-dimension quality evaluator covering relevance, exploitability, completeness, precision, safety, and correctness. Finally, the accepted records are exported as SFT data for the public release. The implementation, including source adapters, prompts, class definitions, and audit files, is released alongside the dataset.

The released run starts from 9,964 curated CVEs and produces 6,159 SFT records, corresponding to a 61.8% end-to-end yield. The accepted records cover 149 named CWE classes plus a catch-all CWE-Other bucket and 1,983 source-project strings. Their mean PoC quality score is 27.90 out of 30 under the six-dimension rubric, and a cross-judge replay with three independent LLM families (Gemini-3.1-Pro, Claude-Opus-4.6, and GPT-5.4) places the dataset in the same high band as the construction-time judge. Fine-tuning a 9B Qwen3.5 student on RealMythosReasoning reaches 24.6/30 on a 141-CVE held-out benchmark, within 1 point of the 3× larger qwen3.6 (27B, 25.6) and the frontier closed API deepseek-v4-flash (25.1), while the same untuned 9B baseline produces only 3 parsable answers. These results show that the release is not only a collection of vulnerability labels, but a quality-controlled corpus of real-code reasoning records that transfers to downstream PoC generation.

In summary, this paper makes the following contributions:

- We identify a concrete data gap in cybersecurity reasoning: public resources rarely connect real vulnerable code, reasoning traces, and PoC-oriented outputs in a form suitable for supervised fine-tuning.
- We propose a dataset-construction framework that uses privileged teacher-side evidence for generation while releasing patch-unaware student-side reasoning for training.
- We present RealMythosReasoning, a 6,159-record dataset with documented coverage and quality signals, as a first step toward open and reproducible security reasoning data.
- We show that fine-tuning a 9B open model on RealMythosReasoning matches a 3× larger open model and approaches a frontier closed API on unseen CVEs, demonstrating that the released corpus transfers to downstream PoC generation.

2 Motivation

2.1 Related Work

As large language models grow more capable, their cyber-offensive and -defensive abilities have become a focal point of the research community. Frontier closed systems such as Anthropic’s *Mythos* [3], OpenAI’s GPT-5.5 [19], and Google’s *Big Sleep* [6] agent can already locate, explain, and exploit real C/C++ vulnerabilities at near-expert quality, with Big Sleep alone disclosing more than twenty novel CVEs in widely deployed open-source software in 2025. Yet their weights, training data, and access remain private. Closing this open-vs-closed gap, so that public models perform reliably in real vulnerability discovery and exploitation, is therefore an increasingly urgent problem. The distillation literature in adjacent domains [1, 12, 13] converges on a single answer: the most effective lever is supervised fine-tuning on a high-quality, domain-matched dataset. The recipe is settled; the asset is not.

The open community’s cybersecurity datasets fall into two groups, and neither alone yields the (real CVE, reasoning trace, runnable PoC) tuple this work targets. The first group is limited to cybersecurity knowledge understanding: Primus-Reasoning [25], CyberPal / SecKnowledge 2.0 [14], SecCoT-CN [8], AquilaX-AI [4], and cybersec-reasoning-merged [17] pair security questions or

concepts with CoT traces, so the student-visible input is a synthesized prompt rather than a real vulnerable function. The second group exposes real CVEs through C/C++ source corpora: PrimeVul [11], MegaVul [18], ReposVul [23], and CVEfixes [5] ship labelled vulnerable functions but no reasoning traces or PoCs (CleanVul [15] joins later via the same adapter), and a systematic audit [9] flags non-trivial label noise that motivates the curation stage of §3.1. Orthogonally, interactive PoC harnesses (CyberGym [24], PatchEval [7], PoCGen [20], CVE2PoC [22], DrillAgent [21], PAGENT [10], PACEbench [16]) and the concurrent SecVulEval [2] benchmark consume the same CVE pool as evaluation environments, not as the offline SFT supervision a student must learn from before entering one. Table 1 cross-tabulates these releases against the five attributes that determine whether a record can serve as patch-unaware, runnable defensive supervision.

Each line above is informative on one axis and silent on the others, so the gap is structural rather than incidental. Group 1 supplies reasoning without real-code grounding: a student trained on synthesized prompts learns to talk about security, not to read a concrete vulnerable function. Group 2 supplies real-code grounding without supervision: vulnerable functions ship bare, with no reasoning, no PoCs, and non-trivial label noise. The orthogonal harnesses score interaction-time output but release no offline (prompt, reasoning, PoC) records to learn from. As a result, no open dataset jointly delivers a real C/C++ CVE function as the student-visible input, a step-by-step reasoning trace, and an executable-shaped proof-of-concept under a per-record quality gate, which is precisely the asset the open-vs-closed gap demands. §2.2 examines why the naive route to build it fails.

2.2 Design Rationale

As shown in §2.1, existing open datasets sit on opposite sides of the same gap: knowledge-understanding releases surface reasoning over synthesized prompts but never see a real vulnerable function, while CVE source corpora ship real vulnerable functions but no reasoning and no PoC. A dataset that genuinely lifts model cybersecurity capability must bridge the two, pairing a real-CVE input with code-grounded reasoning and an executable-shaped PoC under a per-record quality gate.

We therefore organize the dataset around three design dimensions, each restoring one component prior open releases failed to deliver jointly. First, the output is a runnable PoC whose intended behavior is to trigger the underlying CVE, so the model learns to produce exploit code rather than to describe security concepts in prose. Second, the input is a real C/C++ CVE function drawn from public source corpora, so reasoning is exercised over real vulnerable code rather than a synthesized prompt. Third, the reasoning is a step-by-step trace linking the visible code to the PoC, so the supervision is a derivation rather than an answer key. §3 details the multi-stage pipeline that realizes these three dimensions.

3 RealMythosReasoning: Dataset Construction

This section describes how we construct **RealMythosReasoning**, a security-reasoning corpus derived from real vulnerability records.

Dataset	Size	Teacher	CoT	Real CVE	PoC	Patch-unaw.	Quality gate
Primus-Reasoning [25]	4,864	o1 / R1	✓	✗	✗	✗	✗
Cybersec-Reasoning-Merged [17]	23,146	mixed	✓	✗	✗	✗	✗
AquilaX-AI [4]	18,282	template	✓	✗	✗	✗	✗
SecCoT-CN [8]	31,921	GPT-4.1 / Qwen3	✓	✗	✗	✗	✗
SecKnowledge / 2.0 [14]	153K / 403K	expert + LLM	✓	✗	✗	✗	✗
OpenCodeReasoning [1]	736,712	R1	✓	✗	✗	✗	✗
CVE source corpora [†]	source only	N/A	✗	✓	✗	✗	✗
RealMythosReasoning (this work)	6,159	DeepSeek-V4-Pro	✓	✓	✓	✓	✓

Table 1: Cyber-reasoning datasets and vulnerability source corpora scored on the five attributes required for patch-unaware, runnable defensive supervision. *CoT*: step-by-step trace; *Real CVE*: real C/C++ vulnerable function as student input; *PoC*: per-record proof-of-concept under the six-axis rubric of §3.5; *Patch-unaware*: student input free of patch, CVE, or diff text; *Quality gate*: per-record auditable score. [†]PrimeVul, MegaVul, ReposVul, and CVEfixes, used as the canonical curation input (§3.1).

The construction has five conceptual stages (Figure 1): *source curation*, *task construction*, *teacher-guided reasoning generation*, *patch-unaware rewriting*, and *quality-controlled SFT materialization*. The design separates *teacher-only evidence* from the student-visible task. The teacher may use auxiliary information such as the patch, CVE description, CWE label, and augmented context to localize the bug, while the final student prompt exposes only the vulnerable code and the task instruction. On the curated input of 9,964 CVE-unique C/C++ vulnerabilities, the pipeline accepts 6,159 SFT records, corresponding to a 61.8% end-to-end yield.

3.1 Source Curation and Canonicalization

RealMythosReasoning starts from four open vulnerability corpora: PrimeVul [11], MegaVul [18], ReposVul [23], and CVEfixes [5]. During source selection, we reviewed public vulnerability datasets covering 2017–2025 and retained the datasets that best satisfy three practical requirements: clear CVE-based organization, sufficient scale, and active maintenance. We focus on C/C++ vulnerabilities because they provide abundant real-world memory-safety, input-validation, and parser-failure cases, and because vulnerable / fixed function pairs are often recoverable from public patches. Each source is normalized into a common record containing the CVE identifier, CWE label, project metadata, CVE description, vulnerable function, fixed function, source name, and raw source payload. All later stages read this common representation, making source-specific file formats irrelevant after ingestion.

The curation stage addresses two common problems in vulnerability datasets: duplicate examples and inconsistent source granularity. Sources are visited in a fixed priority order, and the first accepted record for each CVE is retained. Later occurrences of the same CVE are logged for audit but do not enter the training pipeline. This first-source-wins rule favors manually curated vulnerable / fixed pairs when they are available, while still allowing lower-priority commit- or hunk-derived corpora to expand coverage. The resulting curated input contains 9,964 unique CVEs (Table 2).

3.2 Reasoning Task Construction

After curation, each record is converted into a reasoning task. The central difficulty is that a single vulnerable function is often not self-contained: its behavior may depend on external structs, macros,

Source	Priority	CVEs	Share
PrimeVul	100	3,264	32.8%
MegaVul	80	6,215	62.4%
ReposVul	70	67	0.7%
CVEfixes	60	418	4.2%
Total		9,964	100%

Table 2: Curated input after priority-based CVE deduplication over the selected 2017–2025 vulnerability sources. CleanVul is supported by the source adapter but was added after the main run and is therefore not included in these counts.

helper functions, protocol fields, or calling conventions. We therefore use a lightweight context-augmentation procedure to identify missing dependencies and attach only the minimal declarations required to make the function analyzable. This step is not intended to explain the vulnerability; it supplies the syntactic and semantic context needed for a model to reason about the visible code.

The task is also routed to a vulnerability-class template. A CWE-normalized router maps each case into one of seventeen broad classes such as memory bounds, numeric calculation, input validation, filesystem access, cryptographic protocol misuse, race condition, and resource lifetime. Each class specifies the analysis focus and the expected shape of a useful proof-of-concept (PoC). This class-aware prompting is important because different vulnerability families require different evidence: for example, a memory-bound violation typically needs a concrete buffer layout and trigger input, whereas an access control failure requires a privilege boundary and an unauthorized operation. Routing therefore narrows the teacher’s task without collapsing the original CWE metadata, which remains stored in the final record.

3.3 Teacher-Guided Trace Generation

For each constructed task, we ask a frontier teacher model (DeepSeek-V4-Pro with reasoning enabled) to generate both a technical reasoning trace and a PoC. Unlike ordinary supervised data collection, the teacher is given teacher-only evidence: the vulnerable function, fixed function, CVE description, CWE label, augmented context, and class-specific task instruction. The patch and CVE description provide a localization signal, helping the teacher

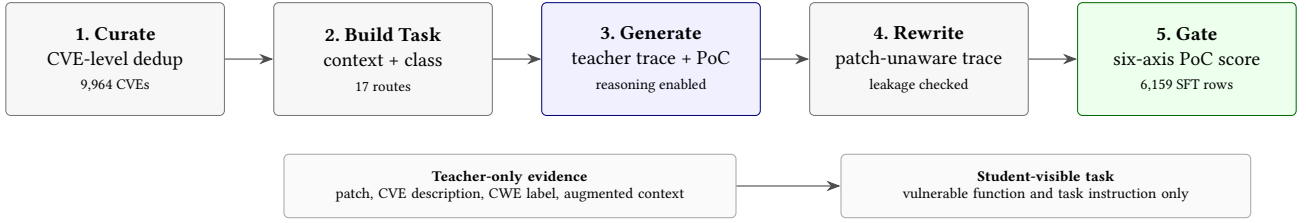


Figure 1: Overview of the RealMythosReasoning construction pipeline. Teacher-only evidence denotes auxiliary information available during trace generation, including the patch, CVE description, CWE label, and augmented context. The released SFT task removes these privileged references and exposes only the vulnerable code and task instruction to the student.

identify the true root cause rather than spending most of its reasoning budget guessing which branch or data flow is vulnerable.

The generation output is treated as an intermediate record rather than final training data. We store the raw teacher trace together with the structured inputs that produced it, so the provenance of each example remains inspectable. This audit record also supports the next stage: reviewers can compare the privileged teacher trace with the student-visible rewrite and check how patch- or CVE-specific statements were removed or re-anchored to the vulnerable code.

3.4 Patch-Unaware Rewriting

The teacher trace is not used directly for SFT. Instead, it is passed through a conservative rewriting stage that converts privileged reasoning into a student-visible explanation. The editor is allowed to remove explicit CVE identifiers, patch references, commit metadata, and sentences that depend only on non-visible evidence. When a patch-derived observation can be grounded in the vulnerable function itself, the editor rewrites it into code-facing language; otherwise, the sentence is deleted. The editor is not allowed to summarize the trace, add new facts, or change the PoC strategy.

This stage creates the key supervision split in RealMythosReasoning. The teacher can use privileged information to construct a high-quality explanation, but the student is trained on a reasoning process that appears to arise from inspecting the vulnerable code alone. Persisting both versions also makes leakage auditable at the sentence level. In practice, this audit trail is essential because patch leakage is not a binary file-level property: it often appears as small phrases that must be removed without damaging the technical reasoning chain.

3.5 Quality Gating and SFT Materialization

The final stage decides whether a rewritten example is sufficiently useful for SFT. We evaluate the generated PoC with a six-axis rubric (Table 3) covering relevance, exploitability, completeness, precision, safety, and correctness. These axes separate failure modes that a single scalar judgment would obscure: a PoC may be syntactically complete but fail to reach the vulnerable sink, or it may target the right function while demonstrating a generic rather than vulnerability-specific behavior. Records are accepted when their total score and per-axis profile pass the acceptance rules; borderline cases can be retained for manual review or regeneration.

In addition to the LLM judge, a deterministic verifier checks structural properties such as the presence of an entry point, a visible trigger action, and basic syntactic consistency. The verifier does

Dimension	Criterion scored on 1–5
Relevance	Targets the specific vulnerability and code-level mechanism.
Exploitability	Reaches the vulnerable behavior through a plausible trigger path.
Completeness	Includes sufficient setup, payload, and trigger context.
Precision	Demonstrates the exact flaw rather than a generic attack.
Safety	Avoids unnecessary destructive behavior beyond demonstration.
Correctness	Is syntactically and logically sound for the target context.

Table 3: Six-axis PoC quality rubric used for corpus acceptance.

not replace semantic judging; rather, it catches format and executability failures that a model judge may tolerate. Combining the LLM rubric with static checks gives users a more transparent acceptance signal and allows them to re-filter the corpus by individual quality dimensions.

Each accepted example is materialized as one JSONL row. The row contains a code-only question, a patch-unaware reasoning trace, the final response with the PoC, per-axis quality scores, and metadata such as project, source corpus, CWE tokens, and routed vulnerability class. The schema is deliberately independent of any particular chat template: trainers may concatenate the reasoning and answer fields for reasoning-style SFT, use the answer alone for direct-response SFT, or apply stricter filtering on a chosen rubric dimension.

3.6 Reproducible Execution

The construction pipeline is designed for long-running, resumable execution. The curated input is split into fixed-size shards, each with a manifest recording the task identifier, record count, source composition, and input hash. Workers validate shard hashes before processing, write per-case results and failure logs, and resume from case-level checkpoints after interruption. Failed cases can be retried without re-running completed examples.

After all shards finish, a strict merge pass validates task completion and combines outputs in deterministic shard order. Given the same per-task outputs, the merge produces a byte-stable training JSONL and a stable corpus hash. The teacher model itself is not guaranteed to be bit-deterministic across future API calls, so reproducibility is defined over the released records and the frozen per-task outputs rather than over re-querying the provider. This distinction is important for open security datasets: users should be

able to audit, filter, and retrain from the released corpus even if the original teacher endpoint changes.

4 Evaluation

4.1 Setup

All teacher, judge, and rewriting calls in the released run use a single model, **DeepSeek-V4-Pro**. Dataset construction (§3) and the analyses below ran on a single server with an Intel Xeon Platinum 8276 CPU and 256 GB of memory. External model calls cost **\$952.18** in total, and every statistic below can be reproduced deterministically from the released analysis script. We validate **RealMythosReasoning** along three intrinsic axes: *quality* (§4.2), *cross-judge agreement* (§4.3), and *coverage* (§4.4).

4.2 Trajectory and Quality Profile

RealMythosReasoning consists of **6,159** supervised fine-tuning trajectories, each pairing a code-only question with a patch-unaware reasoning trace and a PoC response. The average question is 8,938 characters long (median 5,704), the average reasoning trace 3,617 characters (median 2,651), and the average PoC 5,466 characters (median 4,086), yielding a per-trajectory average of 18,022 characters. Trajectory lengths range from 2,273 to 397,573 characters, covering short, medium, and long cases.

We evaluate every trajectory with the six-axis rubric of §3.5. The mean total score is **27.90** out of 30 (median 30, std 3.37), with 50.8% of trajectories at the 30/30 ceiling and 72.5% at or above 28. Only 125 trajectories (2.0%) fall below the nominal acceptance total of 18; we retain them so downstream users can apply stricter thresholds, and their inclusion does not materially shift the headline mean. Five of the six axes saturate (Table 4, Figure 2a): *Safety* and *Completeness* both exceed 4.9 with weak-axis rates below 0.4%, and *Relevance*, *Precision*, and *Correctness* sit between 4.5 and 4.8. The only binding axis is *Exploitability*, at mean 4.09, std 1.33, and weak-axis rate 19.3%. This pattern matches the looks-like-PoC failure mode of §2.2: the judge accepts PoCs that compile and look on-target but vary in how specifically they reach the vulnerable sink. Even on this binding axis the average trajectory exceeds 4/5, so the remaining headroom is concentrated on a single well-localized dimension.

Dimension	Mean	Std	% ≤ 2
Safety	4.98	0.19	0.1%
Completeness	4.93	0.34	0.4%
Relevance	4.73	0.83	5.1%
Precision	4.67	0.95	7.2%
Correctness	4.51	0.98	8.8%
Exploitability	4.09	1.33	19.3%

Table 4: Per-dimension statistics on the 6,159-record dataset. Five of the six axes have mean above 4.5; only *Exploitability* binds.

4.3 Cross-Judge Validation

The headline score of §4.2 comes from a single judge, DeepSeek-V4-Pro. To check that this number reflects dataset quality rather than judge-specific calibration, we re-score a fixed 20-case subsample

(uniform random, seed 114; 0.32% of the release) with three additional LLM judges of different provider families: Gemini-3.1-Pro, Claude-Opus-4.6, and GPT-5.4 (Table 5, Figure 2b). All four judges place the dataset in a high band. Gemini-3.1-Pro tracks DeepSeek-V4-Pro within 0.6 total points and shares the same median of 29/30. Claude-Opus-4.6 shifts the range down by about 4 points while preserving the per-record ordering, and its median of 24/30 still clears the nominal acceptance threshold of 18. GPT-5.4 is the strictest judge, but its gap is concentrated on a single axis: mean *Correctness* is 1.9 under GPT-5.4 versus 3.2 to 4.7 under the other three, and removing that axis collapses its band onto Claude-Opus-4.6. Aggregated over the four judges, dataset quality agrees on five of the six rubric dimensions, so the headline of 27.90/30 is not specific to the judge used at construction time.¹

Judge ($n=20$)	Mean	Med.	Min	Max
DeepSeek-V4-Pro	28.1	29	18	30
Gemini-3.1-Pro	27.5	29	21	30
Claude-Opus-4.6	23.6	24	18	29
GPT-5.4	19.2	20	13	24

Table 5: Cross-judge totals on a fixed 20-case subsample. All four judges place the median at or above the nominal acceptance threshold of 18.

4.4 Coverage

The 6,159-trajectory release spans **149** distinct primary CWE tokens. The ten most frequent classes together account for 62.7% of the dataset (Table 6), and the remaining 37.3% is distributed across 139 further CWE labels, so the tail is broad rather than dominated by a few outliers. The mix combines common memory-safety classes (CWE-119, CWE-125, CWE-787) with input-validation, information-exposure, and resource-management failures (CWE-20, CWE-200, CWE-399). Quality varies systematically with family: memory-bounds classes receive the highest *Exploitability* scores, while input-validation and resource-management cases are correspondingly harder for the teacher to convert into concrete PoCs.

#	CWE	n	Exp. μ	%Exp ≤ 2
1	CWE-119 mem-bounds	654	4.31	14.1%
2	CWE-125 OOB read	626	4.55	9.4%
3	CWE-787 OOB write	527	4.35	14.0%
4	CWE-476 NULL deref	468	4.21	17.3%
5	CWE-20 input valid.	410	3.96	22.4%
6	CWE-190 int overflow	275	3.80	25.5%
7	CWE-Other (un-typed)	262	4.04	20.2%
8	CWE-416 use-after-free	259	3.94	20.5%
9	CWE-200 info exposure	215	3.55	30.7%
10	CWE-399 resource mgmt	166	3.83	27.1%
Top-10 cum.		3,862		(62.7%)

Table 6: Top-10 CWE distribution on the 6,159-record dataset, with per-class *Exploitability* mean and weak-axis rate.

¹Per-case judge outputs can be regenerated from the released evaluation script; a full-dataset four-judge replay would require about 18,500 further judge calls.

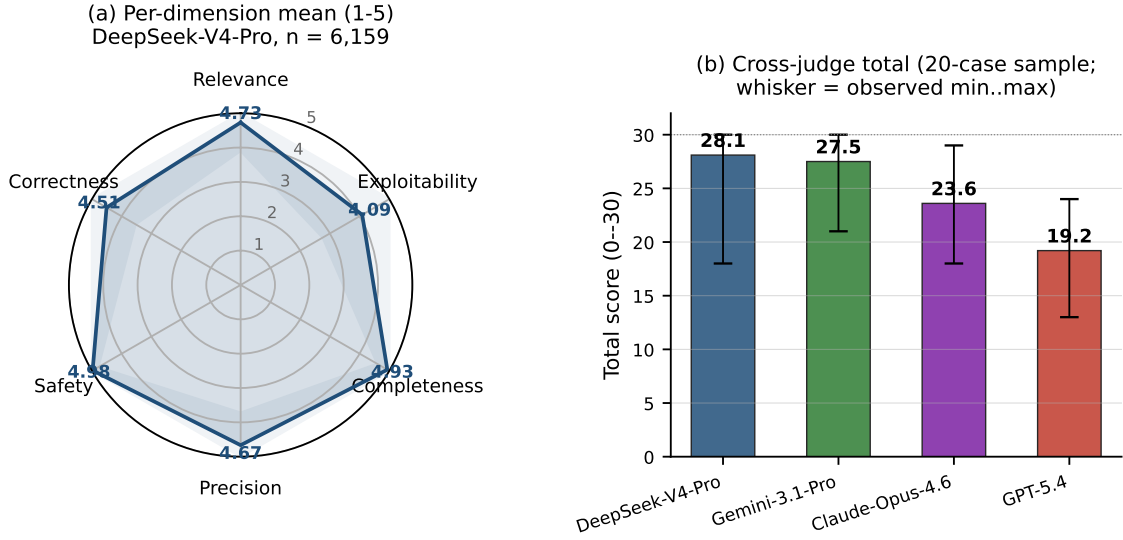


Figure 2: (a) Per-dimension mean (1–5) of the DeepSeek-V4-Pro judge over the 6,159-record dataset; the light band shows one standard deviation and the red dot marks *Exploitability*, the only binding axis. (b) Cross-judge totals on the 20-case subsample; whiskers show observed min and max.

Source coverage is similarly broad, and per-source quality is balanced. MegaVul and PrimeVul contribute the bulk of accepted trajectories (3,445 and 2,400 records), with CVEfixes and ReposVul adding 266 and 48. Total scores cluster tightly across all four sources; the only material per-source difference is on *Exploitability*, where MegaVul cases are slightly easier to convert into concrete PoCs than those from the other three sources. At the project level, the release spans **1,983** distinct project strings, most of which appear only once, with the top ten (led by linux, torvalds/linux, tensorflow/tensorflow, wireshark/wireshark, and gpac/gpac) covering 30.0% of the dataset. A handful of these entries are raw-string variants of the same repository, and merging the aliases would still leave the long tail dominant. Together, the CWE, source, and project distributions support both holistic use of the dataset and fine-grained selection by class, repository, or vulnerability family.

5 Downstream Fine-Tuning Effectiveness

Intrinsic quality (§4) does not guarantee transfer. The sharper question is whether a small student fine-tuned on RealMythosReasoning can generate working PoCs for unseen vulnerabilities. We answer it with a held-out benchmark and a head-to-head comparison against open and closed baselines.

5.1 Setup

Test set. We hold out 284 records from the DiverseVul reserved split, disjoint from all training data, and run each through the same context-augmentation procedure used in dataset construction. deepseek-v4-pro checks whether the vulnerable function is self-contained and supplies missing structs, macros, signatures, and call harnesses when it is not; a second pass verifies sufficiency over up to three feedback rounds, retaining 175 records. To curb single-judge bias, gemini-3.1-pro re-verifies each context against the

same standard, and only records both judges accept enter the final set. The resulting benchmark spans 141 CVEs across 65 projects and 36 CWE categories. Each prompt redacts the CVE identifier and description; the metadata is kept only as ground truth for scoring. **Models.** We fine-tune a Qwen3.5-9B student on RealMythosReasoning and denote it **qwen3.5-9B-RM**. Two same-scale open backbones, qwen3.5-9B and llama-3.1-8B, act as untrained baselines. Two stronger systems set upper references: qwen3.6 (27B), a 3× larger general-purpose open model, and deepseek-v4-flash, a frontier closed API. The question is whether a compact 9B student trained on our trajectories can match the larger open model and approach the closed API on unseen vulnerabilities.

Metric. All systems run with thinking enabled, temperature 0.0, and a 16,384-token budget, producing one final answer per record. deepseek-v4-pro scores each answer on the six-axis rubric of §3.5 (relevance, exploitability, completeness, precision, safety, correctness), each axis rated 1 to 5 for a maximum of 30. Empty final answers are counted as missing generations and excluded from the scored set.

5.2 Results

Blind prompting creates a known failure mode: a model may identify a real but *off-target* vulnerability in the snippet and still score low against the official CVE label. To avoid penalizing genuine analysis, every response with raw score ≤ 15 is adjudicated by a separate deepseek-v4-pro call. Responses labelled VALID (a plausible real finding) and truncated evaluator outputs are excluded from the filtered mean; INVALID responses are retained as legitimate low scores.

Table 7 reports the outcome. Without fine-tuning, the same-scale open baselines collapse: qwen3.5-9B and llama-3.1-8B return parsable PoCs on only 3 and 2 of 141 records, and score in the low teens on those few. Fine-tuning the same Qwen3.5-9B backbone on RealMythosReasoning lifts the parsable rate from 2% to 81% and

Table 7: Downstream PoC quality on the 141-record held-out test set. *Eval.*: records with a parsable final answer. *Raw*: mean over evaluated records. *Filtered*: mean after removing **VALID off-target findings (raw ≤ 15) and truncated evaluator outputs; maximum is 30. Untrained baselines fail to produce parsable PoCs (≤ 3 of 141), so their filtered means are omitted.**

Model	Eval./141	Raw	Filtered
qwen3.6 (27B)	139	24.0	25.6
deepseek-v4-flash	138	22.9	25.1
qwen3.5-9B-RM (ours)	114	23.8	24.6
qwen3.5-9B (no fine-tune)	3	11.7	n/a
llama-3.1-8B	2	9.0	n/a

the filtered mean to **24.6/30**, landing within 1 point of the $3\times$ larger qwen3.6 (27B) and within 0.5 of the frontier deepseek-v4-flash API. A compact 9B student trained on our trajectories therefore closes most of the gap to systems an order of magnitude larger or hidden behind closed APIs.

6 Conclusion

This paper presented **RealMythosReasoning**, a real-world cybersecurity reasoning corpus that pairs vulnerable C/C++ code with patch-unaware reasoning traces and quality-gated PoCs. The construction pipeline combines CVE-level source curation, lightweight context augmentation, class-aware prompting, teacher-guided trace generation, patch-unaware rewriting, and six-axis PoC quality assessment. On 9,964 CVE-unique inputs, the pipeline produces 6,159 SFT-ready records spanning diverse CWE and project families. Our intrinsic analysis shows that the released corpus is broadly well-formed under the proposed rubric, while also identifying exploitability and executable validation as the main remaining bottlenecks. We hope this release provides a transparent foundation for open, inspectable, and reproducible cybersecurity reasoning research.

References

- [1] Wasi Uddin Ahmad, Sean Narenthiran, Somshubra Majumdar, Aleksander Ficek, Siddhartha Jain, Jocelyn Huang, Vahid Noroozi, and Boris Ginsburg. 2025. OpenCodeReasoning: Advancing Data Distillation for Competitive Coding. In *Conference on Language Modeling (COLM)*. arXiv:2504.01943.
- [2] Md Basim Uddin Ahmed, Nima Shiri Harzevili, Jiho Shin, Hung Viet Pham, and Song Wang. 2025. SecVulEval: Benchmarking LLMs for Real-World C/C++ Vulnerability Detection. *arXiv preprint* (2025). arXiv:2505.19828.
- [3] Anthropic. 2026. Project Glasswing: Securing Critical Software for the AI Era. <https://www.anthropic.com/glasswing>. Describes Claude Mythos Preview and Project Glasswing.
- [4] AquilaX-AI. 2025. AquilaX-AI Security-Assistant Reasoning Dataset. Hugging Face dataset, <https://huggingface.co/datasets/tuandunghcmut/AquilaX-AI-security-assistant-reasoning>. 18,282 reasoning samples for vulnerability analysis and secure coding.
- [5] Guru Bhandari, Amara Naseer, and Leon Moonen. 2021. CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software. In *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE)*. arXiv:2107.08760.
- [6] Big Sleep Team, Google Project Zero and Google DeepMind. 2024. From Naptime to Big Sleep: Using Large Language Models to Catch Vulnerabilities in Real-World Code. <https://projectzero.google/2024/10/from-naptime-to-big-sleep.html>. Subsequent 2025 disclosures report 20+ novel vulnerabilities found by Big Sleep in open-source projects (e.g., CVE-2025-6965 in SQLite).
- [7] ByteDance Security Lab. 2025. PatchEval: A New Benchmark for Evaluating LLMs on Patching Real-World Vulnerabilities. *arXiv preprint arXiv:2511.11019* (2025).
- [8] cfrylhy. 2025. SecCoT-CN: Chinese Network-Security Chain-of-Thought Dataset. Hugging Face dataset, <https://huggingface.co/datasets/cfrylhy/SecCoT-CN>. 31,921 question-reasoning-answer triples; CC-BY-NC-ND-4.0.
- [9] Roland Croft, M. Ali Babar, and M. Mehdi Kholoosi. 2023. Data Quality for Software Vulnerability Datasets. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. arXiv:2301.05456.
- [10] Achintya Desai, Md Shafiuazzaman, Wenbo Guo, and Tefvik Bultan. 2026. PAGENT: Program-Analysis-Guided LLM Agent for Proof-of-Concept Generation. *arXiv preprint arXiv:2604.07624* (2026).
- [11] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. 2025. Vulnerability Detection with Code Language Models: How Far Are We?. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*. arXiv:2403.18624.
- [12] Etash Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Sprague, et al. 2025. OpenThoughts: Data Recipes for Reasoning Models. *arXiv preprint arXiv:2506.04178* (2025).
- [13] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. 2025. TULU 3: Pushing Frontiers in Open Language Model Post-Training. In *Conference on Language Modeling (COLM)*. arXiv:2411.15124.
- [14] Matan Levi, Yair Allouche, Daniel Ohayon, and Anton Puzanov. 2025. CyberPal.AI: Empowering LLMs with Expert-Driven Cybersecurity Instructions. *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 23. arXiv:2408.09304; companion dataset *SecKnowledge/SecKnowledge 2.0*.
- [15] Yikun Li, Ting Zhang, Ratnadira Widyasari, Yan Naing Tun, Huu Hung Nguyen, Tan Bui, Ivana Clairine Irsan, Yiran Cheng, Xiang Lan, Han Wei Ang, et al. 2024. CleanVul: Automatic Function-Level Vulnerability Detection in Code Commits Using LLM Heuristics. *arXiv preprint arXiv:2411.17274* (2024).
- [16] Zicheng Liu, Lige Huang, Jie Zhang, Dongrui Liu, Yuan Tian, and Jing Shao. 2025. PACEbench: A Framework for Evaluating Practical AI Cyber-Exploitation Capabilities. *arXiv preprint* (2025). arXiv:2510.11688.
- [17] Mohammadcse. 2025. Cybersec-Reasoning-Merged: A Cybersecurity Chain-of-Thought Dataset. Hugging Face dataset, <https://huggingface.co/datasets/Mohammadcse/cybersec-reasoning-merged>. 23,146 samples merging AquilaX-AI (18,282) with Primus-Reasoning (4,864).
- [18] Chao Ni, Liyu Shen, Xiaohu Yang, Yan Zhu, and Shaohua Wang. 2024. MegaVul: A C/C++ Vulnerability Dataset with Comprehensive Code Representations. In *Proceedings of the 21st International Conference on Mining Software Repositories (MSR), Data and Tool Showcase Track*. arXiv:2406.12415.
- [19] OpenAI. 2026. GPT-5.5 System Card. <https://deploymentsafety.openai.com/gpt-5-5/cybersecurity>. Rates GPT-5.5 as *High* on cyber capability under the Preparedness Framework.
- [20] Deniz Simsek, Aryaz Eghbali, and Michael Pradel. 2025. PoCGen: Generating Proof-of-Concept Exploits for Vulnerabilities in Npm Packages. *arXiv preprint arXiv:2506.04962* (2025).
- [21] UIUC Kang Lab. 2026. Execution-State-Aware LLM Reasoning for Automated Proof-of-Vulnerability Generation. *arXiv preprint arXiv:2602.13574* (2026). The agentic framework is referred to as DrillAgent.
- [22] Haoyu Wang, Yanjie Zhao, et al. 2026. CVE2PoC: A Dual-Loop Agent Framework for Proof-of-Concept Generation from CVE Descriptions. *arXiv preprint arXiv:2602.05721* (2026). Huazhong University of Science and Technology.
- [23] Xincheng Wang, Ruida Hu, Cuiyun Gao, Xin-Cheng Wen, Yujia Chen, and Qing Liao. 2024. ReposVul: A Repository-Level High-Quality Vulnerability Dataset. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE), Industry Challenge Track*. arXiv:2401.13169.
- [24] Zhun Wang, Tianneng Shi, Jingxuan He, Matthew Cai, Jialin Zhang, and Dawn Song. 2025. CyberGym: Evaluating AI Agents' Real-World Cybersecurity Capabilities at Scale. *arXiv preprint arXiv:2506.02548* (2025).
- [25] Yao-Ching Yu, Tsun-Han Chiang, Cheng-Wei Tsai, Chien-Ming Huang, and Wen-Kwang Tsao. 2025. Primus: A Pioneering Collection of Open-Source Datasets for Cybersecurity LLM Training. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. arXiv:2502.11191.

A Roadmap

RealMythos is planned as a staged open initiative: **Stage 1** releases RealMythosReasoning, including the curated reasoning data, the construction pipeline, and the intrinsic quality analysis reported in this paper; **Stage 2** will train and evaluate open cybersecurity reasoning models on this data, with comparisons against public

reasoning baselines; **Stage 3** will connect the data and models to reproducible vulnerability environments so that PoCs can be validated against executable targets rather than judged only by textual rubrics; and **Stage 4** will build multi-agent trace collection and verification infrastructure, enabling the community to generate, audit, reproduce, and improve cybersecurity reasoning traces in a fully open workflow.